

School of Electrical, Electronic and Mechanical Engineering

EEME30001 - Embedded Computing Systems

Coursework Description - 2026/27

A Real-Time Embedded Control System

Assessment Brief

Unit Director: Prof. George Oikonomou

v1.2 – October 2026

Contents

1 Introduction **2**

2 Coursework Task **2**

3 Use of Artificial Intelligence **2**
- 3.1 Appropriate and Professional Use of AI . . . 3
- 3.2 Sampled Post-Assessment Vivas . . . 4

4 What You Are Being Assessed On **4**

5 System Architecture Requirement **5**

6 Expected Outcome **6**

7 Functional Specification **6**
- 7.1 Idle State . . . 6
- 7.2 Configuration State . . . 7
- 7.3 Run State . . . 7
- 7.4 Diagnostic State . . . 8

8 Design Guidance **8**

9 Evidence Requirements **9**
- 9.1 Required Evidence 1: Behavioural Evidence . . . 9
- 9.2 Required Evidence 2: Timing-Related Measurement . . . 9
- 9.3 Required Evidence 3: Defect Log . . . 9

10 Report **10**
- 10.1 Length . . . 10
- 10.2 Suggested Structure . . . 10

11 Submission **11**

12 Expectations by Classification **13**

13 Computer-Assisted Marking Support **14**

14 Frequently Asked Questions **15**

15 Final Remarks **17**

Assessment Brief

1 Introduction

The aim of this coursework is to design, implement, test and evaluate a real-time embedded control system using the Texas Instruments LaunchXL-F28379D development board and Code Composer Studio (CCS).

The coursework provides an opportunity to integrate concepts studied throughout the unit, including:

- Embedded C programming
- GPIO
- Timers
- Interrupts
- PWM
- DMA
- Software architecture
- Testing and debugging

The assessment addresses all unit learning outcomes. You are expected to demonstrate engineering judgement in the design and implementation of an embedded system rather than simply produce a working program.

2 Coursework Task

> *Design and implement a real-time embedded control system that satisfies the specification described in this document.*

The system shall be implemented on the LaunchXL-F28379D development board using the C programming language and Code Composer Studio.

The coursework is intended to assess embedded-systems engineering rather than programming effort. A concise, well-structured and well-justified solution is likely to achieve a higher mark than a larger solution that provides little evidence of design, testing, evaluation and engineering judgement.

If you wish, you may interpret the system as forming part of a plausible real-world application of your choosing. You will not need to implement this application in full, but you may use the assumed application context to motivate and justify engineering decisions made throughout the coursework.

3 Use of Artificial Intelligence

This assessment is designated as **Category 3 – Selective** under the University's guidance on the use of Artificial Intelligence (AI) in assessment.

You may choose to use AI tools during the development of your coursework. You may use AI to:

- Write C source code.
- Comment on C source code that you have written yourself.
- Review, edit and refine source code that you have written yourself.

- Write sections of your report.
- Comment on your report.
- Review, edit and refine sections of your report.

You may choose to use any AI tool for any or all of the above tasks, but you do not have to. You will not be advantaged by using AI. You will not be penalised for using AI. If you choose not to use AI, you will not be unfairly disadvantaged compared to students who do use AI.

You will not receive a higher mark by writing more lines of code.

Using AI will not automatically result in a higher mark.

> You remain fully responsible for the correctness and quality of all submitted work.

Marks are awarded for demonstrating attainment of the unit learning outcomes and for evidencing sound engineering judgement, effective design decisions, testing, evaluation and technical understanding.

As part of the submission process, you will be required to complete a separate AI Usage Declaration through Blackboard Ultra.

Completion of the AI Usage Declaration is a mandatory component of the submission process.

You must complete and submit the declaration regardless of whether you used AI during the development of your coursework.

If you did not use AI, you must indicate this within the declaration.

Failure to submit the AI Usage Declaration will not be interpreted as evidence that AI was not used.

3.1 Appropriate and Professional Use of AI

The use of Artificial Intelligence tools is permitted within this assessment.

> As with any engineering tool, you remain responsible for the work that you submit.

Appropriate and professional use of AI involves exercising engineering judgement, verifying outputs, and ensuring that all submitted work accurately reflects your attainment of the learning outcomes.

Examples of Appropriate Use

The examples below are illustrative only.

- AI generates a PWM implementation. You test the code, you fix defects, you verify correct operation and can explain how the implementation works.
- AI drafts part of the report. You review the text, correct inaccuracies, incorporate project-specific information and ensure that the report accurately reflects the implemented system.
- AI suggests the use of DMA or another implementation approach. You evaluate the suggestion, decide whether it is appropriate and you can explain the reasoning behind the decision.

Examples of Inappropriate Use

The examples below are illustrative only.

- Submitting AI-generated source code that you did not review before submission, that you do not understand, and that you cannot explain or discuss.
- Working in a group to generate the source code and a report and then using those for multiple individual submissions. This is individual coursework and collaboration is not allowed.
- Presenting fabricated testing evidence, measured values or experimental results.
- Claiming that measurements were taken from hardware when they were not.
- Knowingly submitting AI-generated outputs that have not been verified for correctness and suitability.

Please be aware that:

i) You may be required to justify and demonstrate genuine understanding of work that you have submitted; and
ii) Inaccurate or misleading declarations regarding the use of AI may result in the matter being considered under the University's academic misconduct procedures.

3.2 Sampled Post-Assessment Vivas

In order to provide additional assurance that submitted work reflects the attainment demonstrated by the student, a random sample of assessment submissions may be selected for a short post-assessment viva. Students selected for a viva may be asked to discuss aspects of their submitted work and demonstrate understanding of the technical decisions, implementation approaches and evidence presented in their submission.

A viva is not a standard component of the assessment. Most students will not be selected.

These vivas will take place after the assessment submission. Students selected for a viva will be required to attend an in-person interview so they can discuss their submission with members of the unit team. The interview panel, consisting of two academic members of staff, will ask questions to determine the level of familiarity with and depth of understanding of the submission, and the materials and processes used to create it.

What to expect

A post-assessment viva is nothing to worry about. You simply need to talk through the work that you have done and it's great practice for future interviews or project vivas. You don't need to bring anything with you, but you may find it beneficial to bring your laptop, or any notes you used to create your assessment, to help you explain your process.

If the post-assessment viva panel detects any irregularities or academic misconduct after academic review, the case is referred to the appropriate procedure for investigation and/or consideration, as set out in the University's Academic Misconduct Procedure[1]. To ensure you are meeting all University guidelines, we recommend you read your assessment brief fully and ensure you remind yourself of the University's policy on academic integrity here: Academic Integrity[2].

4 What You Are Being Assessed On

This coursework assesses all unit learning outcomes.

[1] University Academic Misconduct Procedure
[2] University Academic Integrity Guidance

By completing this coursework, you will have the opportunity to demonstrate attainment of the following learning outcomes:

- Demonstrate a comprehensive understanding of embedded computer operational principles, illustrating how they can be effectively integrated into systems for control and monitoring.
- Integrate hardware and software components into a functional embedded system, conducting tests and debugging to ensure proper functionality.
- Implement appropriate interfacing techniques and communication methods during the development of embedded systems. Apply effective testing and debugging methods.
- Design complex embedded systems, taking into consideration real-time requirements and performance parameters in resource- and input/output-constrained environments.

In practical terms, you are being assessed on your ability to understand embedded-system operation, integrate hardware and software into a functioning system, test and evaluate your work, and make and justify engineering decisions.

The assessment is not intended to reward the largest software project, the longest report or the greatest number of source-code files.

Higher marks are achieved by providing stronger evidence of learning-outcome attainment, deeper technical understanding, more effective engineering judgement and more thoughtful evaluation of design decisions.

5 System Architecture Requirement

The system shall be implemented using a state-machine architecture. A minimum of four operational states shall be implemented:

- Idle
- Configuration
- Run
- Diagnostic

You are responsible for:

- Defining state behaviours.
- Implementing state transitions.
- Documenting the state machine.
- Justifying design decisions.

The unit does not explicitly teach state-machine implementation. You are expected to investigate appropriate state-machine techniques independently and apply them appropriately within your design. The diagram illustrates the minimum state structure that shall be implemented. The precise implementation of state behaviours and transitions is left to you.

A user-input mechanism shall be provided to implement transitions between states. User input may be be implemented using the serial inteface, Switch S1 on the Launchpad, or an external trigger switch. If you wish, you may also implement a mechanism where specific transitions occur automatically following a timeout period.

The specific implementation is left to you.

The diagram should be interpreted as a minimum architectural requirement rather than a complete implementation specification. You are encouraged to consider alternative transition paths,

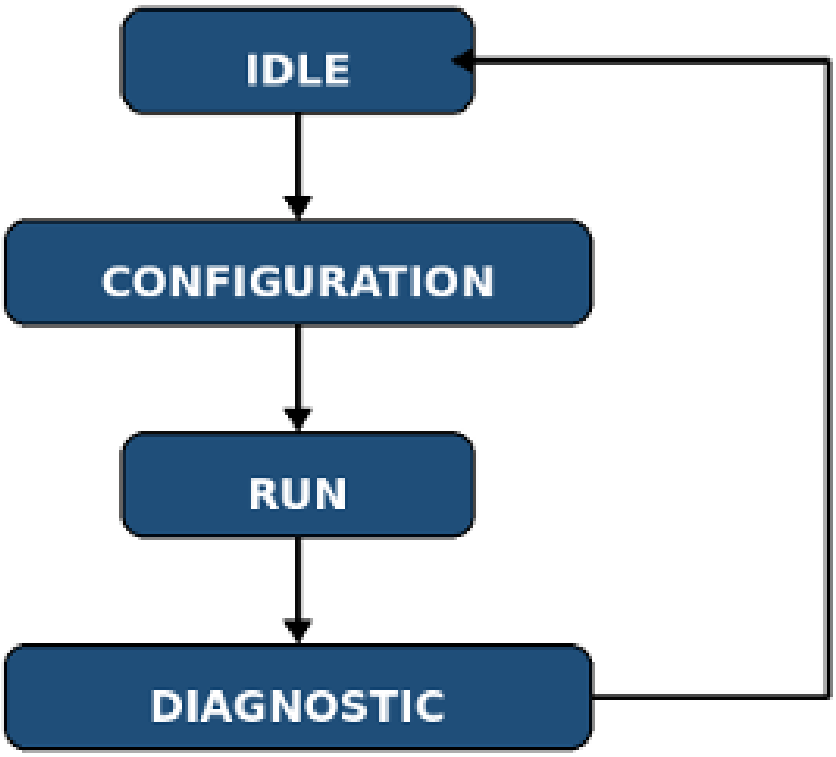


Figure 1: Minimum Required State Structure

additional states and additional functionality where these support your overall design.

> Evidence of independent learning and application of software-engineering approaches will be rewarded.

6 Expected Outcome

Your submission is expected to represent a functioning embedded system.

Marks are awarded for attainment of the learning outcomes rather than completion of individual checklist items. However, attainment is normally evidenced through implementation, testing, evaluation and technical explanation.

The implemented system should therefore demonstrate the functionality described in this specification and be supported by appropriate testing and evaluation.

7 Functional Specification

7.1 Idle State

The Idle state represents the safe startup condition. Minimum requirements:

- PWM output disabled.
- Visual indication that the system is operational.
- Waiting for user interaction.

A mechanism shall be provided to transition the system from Idle to Configuration mode.

7.2 Configuration State

The Configuration state allows operating parameters to be defined. The following parameters shall be configurable.

Parameter A: Output Profile

Available options:

- Constant
- Triangle
- Breathing

Parameter B: PWM Duty Cycle

Available options:

- 25%
- 50%
- 75%
- 100%

For non-constant profiles, Parameter B should be interpreted as the peak duty cycle associated with the selected output profile.

Parameter C: Update Rate

Available options:

- Slow
- Medium
- Fast

You shall select the specific values associated with each of these three settings. Your report should explain the values selected and justify how they relate to the intended behaviour of the system.

You shall determine how parameter values are represented and communicated to the user.

You shall provide mechanisms that allow the user to:

- Move between parameters.
- Modify parameter values.

7.3 Run State

The Run state represents normal system operation. The system shall:

- Generate a PWM output.
- Operate according to the selected configuration.
- Update behaviour according to the selected Output Profile and Update Rate.

The externally observable behaviour of the system shall depend on the selected Output Profile, PWM Duty Cycle and Update Rate settings as follows:

- **Constant Profile:** The PWM Duty Cycle controls the brightness of the output. The output shall continuously alternate between on and off states. The Update Rate parameter controls the duration for which the output remains on or off before changing state.
- **Triangle Profile:** The output shall repeatedly ramp up and then ramp down. The PWM Duty Cycle parameter defines the peak duty cycle reached during operation. The Update Rate parameter controls how quickly the output moves between successive duty-cycle values.
- **Breathing Profile:** The output shall repeatedly fade in and then fade out in a smooth manner intended to create a breathing effect. The PWM Duty Cycle parameter defines the peak duty cycle reached during operation. The Update Rate parameter controls how quickly the breathing cycle progresses.

You may employ polling-based or interrupt-driven approaches.

You should justify the chosen approach and discuss its implications for responsiveness, complexity and performance.

You are strongly encouraged to utilise timer peripherals, interrupts and DMA where appropriate. Effective use of timers, interrupts and DMA may provide evidence of deeper attainment of the learning outcomes.

7.4 Diagnostic State

The Diagnostic state shall provide information about system operation. The system shall display:

- Current configuration settings.
- System uptime or activity count.
- At least one additional diagnostic metric selected by you.

Examples include:

- Timer event count.
- Interrupt count.
- DMA transfer count.
- State-transition count.

The specific implementation is left to you.

8 Design Guidance

The specification defines minimum system functionality.

You are encouraged to extend the system where appropriate, however please note that additional functionality will not automatically attract higher marks.

> Higher marks are awarded through demonstrating deeper attainment of the learning outcomes, stronger engineering judgement, more effective design decisions and better technical evaluation.

Examples of valuable extensions include:

- Interrupt-driven operation.
- DMA-based functionality.

- Enhanced diagnostics.
- Fault detection and handling.
- Event-driven architectures.
- Performance analysis.

9 Evidence Requirements

To support assessment of the learning outcomes, you must include the following evidence within your report.

9.1 Required Evidence 1: Behavioural Evidence

Provide measured evidence of system behaviour for two named configurations. For each configuration, provide:

- the configuration used;
- the expected behaviour;
- the observed behaviour;
- at least one measured quantity;
- the method used to obtain the evidence; and
- a brief interpretation of the result.

Figures, screenshots, tables or other supporting evidence may be included where helpful but are not required.

The purpose of this requirement is to demonstrate that the system was exercised under different operating conditions and that the observed behaviour was compared with the expected behaviour.

9.2 Required Evidence 2: Timing-Related Measurement

Provide one timing-related measurement obtained from your implemented system. You must report:

- the quantity measured;
- the method used to obtain the measurement;
- the measured value;
- a brief interpretation of the result; and
- how the measurement influenced your design decisions, or how it could influence future improvements to the system.

Figures, screenshots or other supporting evidence may be included where helpful but are not required.

The purpose of this requirement is to demonstrate the ability to obtain, interpret and discuss measurements made on an embedded system. The measured value itself is not expected to match any particular target.

9.3 Required Evidence 3: Defect Log

Provide a defect log containing three defects encountered during development. For each defect, provide:

- Symptom
- Diagnosis Method

- Attempted Fix
- Resolution

The Resolution should indicate whether the defect was:

- Fixed
- Partially Fixed
- Open

The purpose of the defect log is to provide evidence of debugging, problem-solving and engineering judgement during development. You are not assessed on the number of defects encountered or the number successfully fixed.

10 Report

10.1 Length

The maximum length of the report is **6 pages**. The page limit does not include:

- An optional title page.
- Optional references.

If an optional title page contains material that would normally form part of the report, that page will be treated as one of the six report pages.

For your report, use a single-column layout and an 11-point font.

There is no penalty for exceeding the page limit. However, any material appearing beyond the first six pages of the report will not be considered during marking.

The purpose of the report is to provide evidence supporting attainment of the learning outcomes. You are therefore encouraged to focus on the quality and relevance of evidence presented rather than the quantity of material included.

You should also be mindful of how much time and effort you dedicate to developing the report within the context of your overall assessment loading during the assessment period.

10.2 Suggested Structure

The report structure presented below is strongly recommended.

It has been designed specifically to help you present evidence relevant to the assessment criteria and learning outcomes in a clear and consistent format.

You may choose to organise your report differently if you wish. However, you remain responsible for ensuring that all required evidence is included and that evidence of attainment remains visible and easy to identify. A highly unusual structure may make it more difficult for assessors to locate evidence that would otherwise support stronger attainment.

Particular attention should be paid to Section 4 (Testing and Validation), which must contain the required evidence specified in this document.

1. System Overview

Brief description of system functionality. Where appropriate, you may explain how design decisions relate to the intended use of the system. This may include assumptions about expected users, performance requirements, reliability requirements or operating conditions.

2. Software Architecture

Include:

- Block diagram.
- State diagram.
- Module structure.

3. Design and Implementation

Discuss:

- State machine.
- PWM implementation.
- Software architecture.
- Timing mechanisms.
- Use of peripherals.

4. Testing and Validation

Discuss:

- Testing methods;
- Results obtained;
- Debugging activities;
- Evidence supporting correct operation.

The Required Evidence described earlier in this document would normally be presented within this section.

5. Evaluation

Critically evaluate:

- Design decisions.
- Strengths.
- Limitations.
- Potential improvements.

Evaluation should consider not only whether the implementation satisfies the functional requirements of the coursework, but also whether it would be suitable for its intended application environment. Where an application context has been assumed, this section will be your opportunity to reflect upon the strengths and limitations of your design within that context.

11 Submission

Submission will take place through Blackboard Ultra. You must complete all required submission components before the submission deadline.

Component 1: Artificial Intelligence Usage Declaration

You must complete the mandatory AI Usage Declaration form provided through Blackboard Ultra. The purpose of the declaration is to provide transparency regarding the use of AI tools during the development of your coursework.

The submission point for the remaining components will only become visible after you have submitted the AI Usage Declaration.

The AI Usage Declaration is submitted separately from the technical report and source-code submission.

The declaration must be submitted regardless of whether AI tools were used during the development of the coursework. Failure to submit the declaration will not be interpreted as evidence that AI was not used.

> You will not be able to see the submission point for the report and source code before you have submitted the AI Usage Declaration

Component 2: Technical Report and Source Code

After you have submitted the AI Usage Declaration, the submission point for your report and source code will become visible on Blackboard Ultra.

> There is a single submission point on Blackboard Ultra for your report and for your source code. Upload **two separate files** using the same submission point.

Report

Accepted formats are:

- PDF (.pdf)
- Microsoft Word (.docx)

You may use any filename you wish.

CCS Project Source Code

Submit a compressed archive containing the source code and project files required to build your Code Composer Studio project. Acceptable archive formats are:

- .zip
- .7z

Your submitted compressed archive must include:

- all .c files;
- all .h files;
- CCS project files; and
- any additional source files required to build the project.

Generated build artefacts and temporary files must not be submitted. Examples of files to exclude from your submission are:

```
Debug/
Release/
.metadata/
```

The emphasis of the assessment is on the quality, structure and effectiveness of the implementation rather than the quantity of source code produced.

12 Expectations by Classification

The statements below are intended as guidance only. They are not additional assessment criteria and should not be interpreted as guarantees of particular marks. Their purpose is to provide an indication of the characteristics typically associated with different levels of attainment.

Pass Mark: This is a level 6 unit. The pass mark for this unit is 40% for all students, including students registered on MSc programmes.

What Should Normally Fail?

> *"It either doesn't work, doesn't do what it is supposed to do, or doesn't provide enough evidence that it does."*

Failure may arise for a variety of reasons. In some cases the implementation is non-functional or substantially incomplete. In other cases functionality may be present but insufficient evidence is provided to demonstrate attainment of the learning outcomes. Major misunderstandings, significant omissions or a lack of supporting evidence are all capable of leading to failure.

What Should Normally Pass / 40–50% Mark?

> *"There are weaknesses, but there is enough evidence here for the examiner to believe the learning outcome has been attained."*

A passing submission will usually demonstrate evidence of a functioning implementation, a recognisable state-machine architecture, implementation of the required operating states, appropriate use of the supplied hardware platform, some testing and validation, and some explanation of design and implementation decisions.

Such work may be relatively simple and may contain weaknesses in architecture, testing, evaluation or technical depth. Those weaknesses do not prevent attainment provided that sufficient evidence exists to support the conclusion that the learning outcomes have been achieved.

What Should Normally Achieve a 2:2 / 50–60% Mark?

> *"The examiner can follow the design decisions and they mostly make sense. Nothing especially exciting. Nothing seriously wrong either. Certainly good enough."*

Work in this range is generally competent. The principal requirements have been achieved, the implementation functions as intended, testing has taken place and the design decisions are broadly reasonable.

This is not necessarily outstanding work, but neither does it suffer from substantial weaknesses. Many competent and complete submissions are likely to sit comfortably within this range.

What Should Normally Achieve a 2:1 / Merit?

> *"This is well-engineered. The student appears to be in control of the design rather than fighting with it."*

Work at this level demonstrates strong attainment of the learning outcomes. Design decisions are coherent, implementation is effective, testing is meaningful and the overall structure of the system reflects good engineering practice.

Students operating at this level appear to understand not only what they have done but why they have done it. The resulting system feels deliberate rather than accidental, and the report provides credible justification for the principal engineering decisions.

What Should Normally Achieve a First / Distinction?

> *"This is excellent work. The student is not just implementing things; they are thinking like an engineer. The quality comes from the judgement, not from the quantity of features."*

First-class work is characterised by thoughtful engineering judgement. The implementation is normally strong, but implementation quality alone is not sufficient to justify first-class attainment.

Students in this range demonstrate technical insight, convincing evaluation, well-reasoned design choices and clear awareness of limitations, trade-offs and alternative approaches. Excellence comes from the quality of the thinking rather than from implementation volume or feature count.

What Should Normally Achieve a High First / High Distinction?

> *"This feels very close to professional engineering. One could imagine this forming part of a product."*

Work in this range should be exceptional. It demonstrates a level of maturity, technical understanding and engineering judgement substantially beyond what would normally be expected.

Such submissions commonly provide sophisticated evaluation, strong evidence of independent thinking and a highly convincing treatment of constraints, alternatives and design decisions. The use of this band should remain rare.

13 Computer-Assisted Marking Support

The Unit Director will use computer-assisted tools to support the marking process. These tools are intended to support the efficient review of submissions and to assist in identifying information relevant to the published marking criteria.

The use of such tools does not replace academic judgement.

All marks are determined by the Unit Director. Automated systems do not assign marks, determine classifications, or apply penalties.

Automated analysis may be used for purposes including, but not limited to:

- comparing reports across the cohort and generating report-similarity analyses;
- comparing source-code submissions across the cohort and generating source-code similarity analyses;
- identifying unusual patterns, similarities or inconsistencies that may warrant closer review by the Unit Director;
- identifying potential inconsistencies between submitted work and the submitted AI Usage Declaration;

- attempting automated compilation or build verification of submitted software and reporting the outcome;
- extracting information from source-code submissions that would otherwise need to be identified manually, including software architecture, module structure, use of interfaces and implementation features;
- locating specific information within submissions that may be relevant during marking, such as testing evidence, timing measurements, defect logs and discussions of engineering decisions;
- generating cohort-level statistics and summaries;
- generating draft feedback after marks have been determined by the Unit Director.

Some of these tools may use artificial intelligence techniques, while others may use conventional software analysis, information extraction, pattern matching, statistical analysis or similar methods.

The outputs produced by such tools are advisory only. They are used to assist the Unit Director in reviewing submissions and extracting information that would otherwise need to be identified manually during marking.

No automatic penalties are applied under any circumstances. Any concerns identified through automated analysis are reviewed by the Unit Director, who remains solely responsible for all assessment decisions and all final marks.

14 Frequently Asked Questions

Do I have to use interrupts?

No.

A polling-based implementation can satisfy the specification and can achieve a passing mark.

However, where appropriate, the effective use of interrupts may provide evidence of deeper attainment of the learning outcomes.

Do I have to use DMA?

No.

DMA is not a required component of the coursework.

However, DMA is strongly encouraged and may provide opportunities to demonstrate deeper attainment of the learning outcomes and stronger engineering judgement.

Do I have to use AI?

No.

The use of AI is entirely optional.

You may choose to use AI tools, use them only for selected tasks, or not use them at all.

Students who do not use AI will not be disadvantaged.

Will I get a higher mark if I submit more code?

No.

The assessment is not based on the number of lines of code submitted.

A concise, well-structured and well-tested implementation is likely to achieve a higher mark than a larger implementation that provides little evidence of technical understanding, testing, evaluation or engineering judgement.

Will a longer report result in a marks penalty?

No.

However, material appearing beyond the first six pages will not normally be considered during marking.

The purpose of the report is to provide evidence of attainment of the learning outcomes. You are therefore encouraged to focus on presenting the strongest evidence clearly and concisely within the permitted length.

Can I change the font size, font face, word processor margins or other document layout elements to make my content fit within the page limit?

No.

It is advisable to focus more on the strength of evidence supporting attainment of the unit learning outcomes than on report style, formatting or length.

Reports should use an 11-point font and a single-column layout.

Normal variations between common fonts and word-processing software are acceptable, but the page limit must not be circumvented through unusually small fonts, altered margins, compressed spacing or other formatting intended to increase the amount of content that can be included within the permitted length.

The unit teaching team will not be able to review your drafts and to respond to questions about whether your chosen format is acceptable.

If in doubt, use standard document settings and focus on presenting the strongest evidence clearly and concisely.

Do I need to include all of my source code in the report?

No.

Source code is submitted separately as part of the CCS project submission.

Where source code is discussed in the report, only short extracts should be used to illustrate important implementation decisions.

Do I have to follow the suggested report structure?

No.

The report structure is strongly recommended but not mandatory. You will not automatically earn extra marks for following it. There is no automatic penalty for changing it.

You remain responsible for ensuring that all required evidence is included, that evidence of attainment is presented clearly and is easy for assessors to identify.

15 Final Remarks

This coursework is intended to provide an opportunity to demonstrate embedded-systems engineering skills through the development of a complete system.

A successful submission is not necessarily the most complex submission.

Strong submissions typically demonstrate:

- Clear software architecture.
- Appropriate engineering decisions.
- Effective testing and debugging.
- Good technical communication.
- Critical evaluation of design decisions.

You are encouraged to focus on quality, understanding and evidence of learning rather than the quantity of code, number of features or length of the report.